BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC PHÚ XUÂN

VIỆN ĐÀO TẠO QUỐC TẾ TOPICA

★★★




# BÁO CÁO ĐỒ ÁN NHÓM

# QUẢN LÝ DỰ ÁN VÀ NHÂN SỰ

Sinh viên thực hiện:

Nguyễn Văn Chung - NBS2503ITA0041

Hoàng Minh Đạt - 2603ITA0025

Lớp: Lập trình hướng đối tượng

Ngành: CNTT – Đồ họa kỹ thuật số

# MỤC LỤC & TỔNG QUAN

## Nội dung báo cáo và hồ sơ đồ án

| Phần | Nội dung chính |
|---|---|
| Chương 1: Giới thiệu | Bối cảnh, phạm vi, yêu cầu chức năng, quy tắc nghiệp vụ |
| Chương 2: Phân tích & thiết kế | Mô hình lớp, kế thừa, composition/aggregation, interface, Factory & Strategy |
| Chương 3: Cài đặt | Đối chiếu 18 tiêu chí OOP với file/class thực tế, kiểm thử và lưu trữ CSV |
| Chương 4: Kết quả | Ảnh giao diện Console trích từ phiên chạy, bộ dữ liệu và kiểm thử |
| Chương 5: Đánh giá | Điểm mạnh, hạn chế và một thiết kế cần cải thiện |
| Phụ lục | Danh mục file nộp, mốc M0-M5, tài liệu hướng dẫn |

## Quy mô và giới hạn

• **Ngôn ngữ:** Java 17+, giao diện Console, không sử dụng Spring hoặc Hibernate.
• **Bốn nhóm nghiệp vụ:** Nhân sự, dự án, công việc, phân công; có bốn màn thống kê.
• **Lưu trữ dữ liệu:** Tồn tại sau khi thoát nhờ bốn file CSV UTF-8.
• **Nguồn yêu cầu:** Slide 37 trang "Đồ án nhóm" môn Lập trình hướng đối tượng; file DO-AN-NHOM.md được nhắc tới nhưng chưa được cung cấp.

> **Định hướng chấm điểm:** Thiết kế hướng đối tượng (3.0 điểm) + Vận dụng kỹ thuật OOP (3.5 điểm) = **6.5 / 10 điểm**. Vì vậy, ưu tiên UML, tính đúng nguyên tắc OOP và khả năng giải thích mã nguồn khi vấn đáp.

# CHƯƠNG 1. GIỚI THIỆU

## 1.1 Bối cảnh, mục tiêu và phạm vi

Một nhóm phát triển phần mềm nhỏ thường cần lưu danh sách nhân viên thuộc nhiều vai trò, theo dõi dự án, chia công việc và giám sát tiến độ. Việc lưu thủ công bằng nhiều bảng rời rạc gây khó khăn khi kiểm tra ai phụ trách việc gì và trạng thái công việc ở từng dự án.

Đồ án triển khai một chương trình Console bằng Java để mô phỏng nghiệp vụ trên. Phần mềm phục vụ mục tiêu học tập OOP: chia trách nhiệm hợp lý, khai thác tính kế thừa và đa hình, xử lý ngoại lệ, áp dụng mẫu thiết kế và lưu trữ file.

## 1.2 Nhóm người dùng và thực thể

• **Người thao tác:** Người quản lý sử dụng menu trên máy tính, không có chức năng đăng nhập/phân quyền.
• **Employee:** Lớp trừu tượng với ba kiểu con cụ thể: Developer, Tester, Manager.
• **Project:** Chứa thông tin dự án, danh sách Task và các bản ghi ProjectMember.
• **ProjectMember:** Tham chiếu hồ sơ nhân viên độc lập để thể hiện quan hệ Aggregation.

## 1.3 Bốn nhóm chức năng yêu cầu

| Nhóm chức năng | Các thao tác chính |
|---|---|
| Nhân sự | Thêm, sửa, xóa, liệt kê, tìm kiếm nhân viên theo mã, tên và vai trò. |
| Dự án | Tạo, cập nhật, xóa, tìm dự án; đổi trạng thái PLANNED / IN_PROGRESS / COMPLETED. |
| Công việc | Tạo, sửa, xóa, tìm, giao người thực hiện, cập nhật TODO / IN_PROGRESS / DONE. |
| Phân công nhân sự | Gán/gỡ/sửa ngày tham gia, tìm kiếm và đếm thành viên; kiểm tra ràng buộc giao task. |

Bên cạnh các nhóm chức năng chính, chương trình còn tích hợp 4 màn thống kê: Thống kê nhân sự theo vai trò, phần trăm task DONE, ước tính chi phí dự án theo Strategy và số task chưa hoàn thành của từng nhân viên.

## 1.4 Quy tắc nghiệp vụ và điều kiện chất lượng

Các quy tắc nghiệp vụ được triển khai ở tầng Service thay vì xử lý trực tiếp trong menu. Tầng Model giữ tính hợp lệ nội tại của đối tượng thông qua setter có validation.

| Mã | Quy tắc / Dữ liệu đầu vào | Phản ứng chương trình |
|---|---|---|
| BR-01 | ID nhân viên, dự án, task phải duy nhất; email nhân viên không trùng. | Thông báo BusinessRuleException nếu trùng. |
| BR-02 | Ngày kết thúc dự án không được trước ngày bắt đầu. | Thông báo ValidationException. |
| BR-03 | Task chỉ được giao cho nhân viên thuộc dự án. | Từ chối giao nếu chưa có ProjectMember. |
| BR-04 | Task DONE phải có nhân viên phụ trách. | Chặn chuyển trạng thái sai. |
| BR-05 | Dự án COMPLETED cần có task và mọi task phải DONE. | Chặn hoàn thành khi chưa đủ điều kiện. |
| BR-06 | Không xóa nhân viên còn thuộc dự án hoặc có task. | Chặn để tránh tham chiếu mồ côi. |
| BR-07 | Xóa dự án kéo theo task và ProjectMember; nhân | Đúng vòng đời |

| | | |
|---|---|---|
| | viên vẫn tồn tại. | composition/aggregation. |
| BR-08 | Sau khi thay đổi dữ liệu thành công, tự lưu CSV. | Dữ liệu giữ nguyên khi chạy lại. |

**Ràng buộc kỹ thuật chính:**

• JDK 17 trở lên; dùng Java chuẩn. Thư viện JUnit 5 chỉ dùng cho việc kiểm thử, không phải framework chạy ứng dụng.

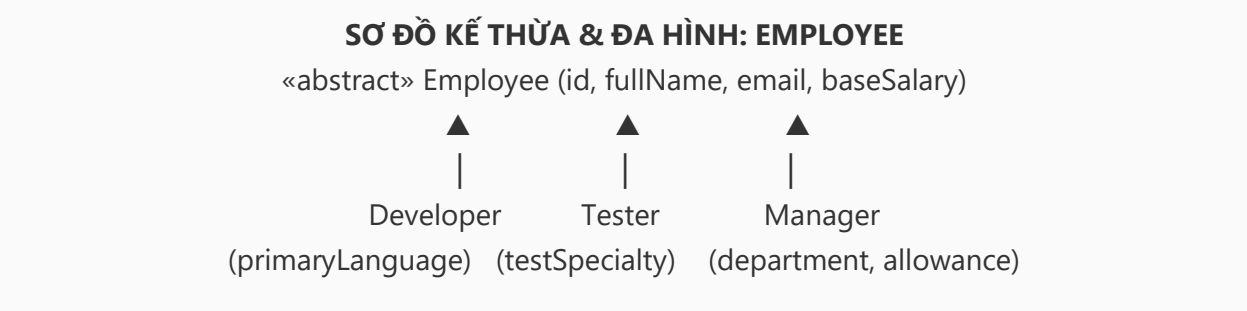
• Ít nhất 2 interface, 1 abstract class, 2 design pattern, 2 custom exception, 2 báo cáo thống kê.

• Menu Console phải xử lý nhập sai, không dừng đột ngột khi người dùng nhập chữ vào ô số, nhập ngày sai hoặc mã không tồn tại.

# CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ

## 2.1 Cây kế thừa nhân sự và đa hình

Lớp Employee gom dữ liệu chung (mã, họ tên, email, lương cơ bản), kiểm tra hợp lệ qua getter/setter và khai báo các phương thức trừu tượng: calculateMonthlyCost(), describeResponsibility(). Các lớp con ghi chồng cả hai phương thức nghiệp vụ để thực hiện cách tính và mô tả công việc phù hợp.




**Lý do chọn kế thừa:**

• Ba vai trò đều là một loại nhân viên: có chung thuộc tính và hành vi cốt lõi; thay đổi cách tính chi phí theo vai trò.

• Danh sách được lưu dưới kiểu Collection<Employee>; ReportService và Strategy xử lý nhiều lớp con qua kiểu cha, khai thác đa hình thật.

• Khi thêm nhân viên mới, chương trình tạo đối tượng qua EmployeeFactory nhưng ReportService không cần viết lại thuật toán.

*Phân biệt:* Abstract class thể hiện quan hệ 'là một loại' và chia sẻ trạng thái chung. Interface Identifiable chỉ nêu hợp đồng định danh, không giữ dữ liệu nhân viên.

## 2.2 Composition, Aggregation và vòng đời đối tượng

| Quan hệ | Lý do lựa chọn |
|---|---|
| Project ◆-- Task (Composition) | Task là thành phần của đúng một Project. Khi xóa Project, Task biến mất khỏi tập dữ liệu theo dự án. |
| Project ◆-- ProjectMember (Composition) | Bản ghi tham gia chỉ tồn tại trong ngữ cảnh một dự án; xóa dự án sẽ xóa các bản ghi tương ứng. |
| ProjectMember ◇-- Employee (Aggregation) | ProjectMember giữ tham chiếu tới Employee có đời sống độc lập; xóa Project không xóa hồ sơ nhân viên. |

Các List được bảo vệ qua Collections.unmodifiableList(); thao tác thêm/xóa phải đi qua các phương thức lớp Project hoặc Service, giảm nguy cơ sửa danh sách sai nghiệp vụ.

## 2.3 Phân lớp trách nhiệm và kiến trúc code

| Tầng | Nhiệm vụ | Lớp tiêu biểu |
|---|---|---|
| Model | Quản lý trạng thái và tính hợp lệ của đối tượng. | Employee, Project, Task |
| Service | Thực thi nghiệp vụ và kiểm tra ràng buộc giữa các đối tượng. | EmployeeService, ProjectService, TaskService |
| Reporting | Tính thống kê; gọi Strategy để thay đổi thuật toán chi phí. | ReportService |
| Storage | Tải/lưu CSV, mã hóa dấu phẩy và dấu nháy kép. | FileDataStore, CsvCodec |
| Presentation | Nhận input, hiển thị lỗi và kết quả, điều phối menu. | ConsoleMenu, ConsoleInput |

Lý do không dùng web hay DB SQL: Slide yêu cầu Java thuần và cho phép Console. File CSV phù hợp việc minh họa OOP, lưu dữ liệu bền vững nhưng không hướng đến môi trường đa người dùng.

## 2.4 Interface, Design Pattern và xử lý ngoại lệ

**Interface và đa hình:**

• Identifiable quy định getId() và cung cấp phương thức hasId() mặc định. Employee, Project, Task cùng tuân thủ một hợp đồng định danh.

• ProjectCostStrategy cung cấp estimate(Project, Map<String, Employee>) và default method details().

| Mẫu | Ý tưởng | Minh chứng |
|---|---|---|
| Factory | Khởi tạo loại Employee tương ứng theo EmployeeType, tránh phân nhánh tạo đối tượng rải rác trong Console. | factory/EmployeeFactory.java |
| Strategy | Cho phép ReportService tính chi phí theo ngân sách | strategy/ProjectCostStrategy.java |

| | | |
|---|---|---|
| | toàn bộ nhân sự 1 tháng hoặc theo effort task được giao, có thể thay thế chiến lược khi chạy. | + 2 implementations |

**Ngoại lệ tự định nghĩa và đúng tầng:**
• ValidationException: Giá trị thuộc tính không hợp lệ (email, lương, giờ, ngày).
• BusinessRuleException: Vi phạm điều kiện nghiệp vụ (trùng ID, nhân viên chưa vào dự án, task chưa DONE).
• EntityNotFoundException: Tìm mã không tồn tại.
• StorageException: File sai cấu trúc hoặc lỗi đọc/ghi; không cố ghi đè dữ liệu hỏng.

ConsoleInput bắt ngoại lệ nhập số/ngày để hỏi lại; Service ném lỗi nghiệp vụ; ConsoleMenu hiển thị thông báo rõ nghĩa. Các lớp lưu trữ ném StorageException có nguyên nhân gốc để chẩn đoán.

# CHƯƠNG 3. CÀI ĐẶT

Vị trí code được ghi tương đối so với src/main/java/vn/pxu/oop/pm/. Mỗi dòng là bằng chứng cần nhóm hiểu và chỉ ra khi vấn đáp.

## 3.1 Đối chiếu 18 yêu cầu kỹ thuật – Mục 01 đến 06

| Mục | Yêu cầu kiểm tra | Vị trí class / method | Minh chứng triển khai |
|---|---|---|---|
| 01 | Số lớp và trách nhiệm rõ ràng | model / service / storage / cli | Các tầng riêng biệt; không đặt nghiệp vụ trong entity hoặc menu. |
| 02 | Thuộc tính private, getter/setter | model/Employee.java; Project.java; Task.java | Các entity encapsulate trạng thái, getter trả view bảo vệ. |
| 03 | Validation trong >=3 setter | Employee.setEmail(); Project.setDates(); Task.setEstimatedHours() | Chặn email sai, ngày ngược, giờ <=0. |
| 04 | >=2 constructor, dùng this() | Employee và lớp con; Project; Task; ProjectMember | Mỗi entity có hai constructor nối chuỗi. |
| 05 | >=2 nhóm nạp chồng phương thức | EmployeeFactory.create(); TaskService.search() | Overload theo bộ tham số; không nhầm với override. |
| 06 | Dùng static / static final có mục đích | Check.EMAIL; TaskEffortCostStrategy.HOURS_PER_MONTH | Pattern regex chung và số giờ chuẩn dùng để ước tính. |

## 3.2 Đối chiếu 18 yêu cầu kỹ thuật – Mục 07 đến 12

| Mục | Yêu cầu kiểm tra | Vị trí class / method | Minh chứng triển khai |
|---|---|---|---|
| 07 | >=2 kết tập, ít nhất 1 composition | Project → Task; Project → ProjectMember; ProjectMember → Employee | Composition xóa theo dự án; Aggregation Employee độc lập. |
| 08 | Kế thừa 1 cha - 3 con | Employee → Developer / Tester / Manager | Ba vai trò dùng lớp cha abstract. |
| 09 | Override toString, equals, >=2 nghiệp vụ | Employee.toString/equals; subclass.calculateMonthlyCost, describeResponsibility | Đối tượng có hành vi khác nhau khi gọi qua Employee. |
| 10 | Abstract class, abstract method | model/Employee.java | Hai method calculateMonthlyCost và describeResponsibility. |
| 11 | >=2 interface và default method | model/Identifiable; strategy/ProjectCostStrategy | Default hasId() và details(). |
| 12 | Đa hình với collection kiểu cha | InMemoryDatabase.employees; ReportService; MonthlyTeamCostStrategy | Collection chứa Developer / Tester / Manager. |

## 3.3 Đối chiếu 18 yêu cầu kỹ thuật – Mục 13 đến 18

| Mục | Yêu cầu kiểm tra | Vị trí class / method | Minh chứng triển khai |
|---|---|---|---|
| 13 | >=2 exception riêng | ValidationException; BusinessRuleException; EntityNotFoundException; StorageException | Phân loại lỗi nhập, nghiệp vụ, tìm kiếm, file. |
| 14 | Xử lý exception đúng tầng | ConsoleInput; ConsoleMenu; FileDataStore; Service | Không catch rỗng trong nghiệp vụ; lớp giao diện thông báo. |
| 15 | >=2 design pattern | EmployeeFactory; ProjectCostStrategy + 2 strategy | Factory + Strategy đều có ngữ cảnh thực tế. |
| 16 | Lưu text/CSV/JSON bền vững | FileDataStore; CsvCodec; data/*.csv | Bốn file UTF-8, save sau thao tác thành công, reload khi chạy. |
| 17 | GitHub >=25 commit, đủ thành viên | Chưa thực hiện - nhóm tự tạo GitHub | Không được giả lịch sử hoặc tác giả commit. |
| 18 | Quy ước Java & code sạch | Toàn bộ package vn.pxu.oop.pm | Tên class PascalCase, method camelCase, package phân lớp. |

## 3.4 Lưu trữ CSV, kiểm tra hợp lệ và cách chạy

| File CSV | Thông tin / Liên kết dữ liệu |
|---|---|
| employees.csv | type, id, fullName, email, baseSalary, detail, extra |
| projects.csv | id, name, description, startDate, endDate, status |
| members.csv | projectId, employeeId, joinedOn (tạo tham chiếu đến Employee đã tải) |
| tasks.csv | id, projectId, title, priority, status, estimatedHours, assigneeId |

**Các lệnh chạy từ thư mục gốc:**

• java -Dfile.encoding=UTF-8 -jar app.jar

• java -Dfile.encoding=UTF-8 -jar app.jar --data-dir data

• mvn test (chạy kiểm thử JUnit 5 trong môi trường Maven)

# CHƯƠNG 4. KẾT QUẢ

## 4.1 Giao diện Console – Nhân sự và Dự án

Dưới đây là bản ghi console được tái hiện từ phiên chạy thực tế (lưu tại docs/RUN_LOG.txt):

```
=== QUẢN LÝ DỰ ÁN VÀ NHÂN SỰ ===
[01] QUẢN LÝ NHÂN SỰ: DANH SÁCH
NV01 | Nguyễn Minh An | Developer | an@example.org | lương 16,000,000 | Java (3 năm)
NV02 | Trần Thu Hà | Tester | ha@example.org | lương 13,000,000 | Kiểm thử Functional tự động
NV03 | Lê Quốc Bình | Manager | binh@example.org | lương 22,000,000 | PMO | phụ cấp 3,500,000
NV04 | Phạm Gia Huy | Developer | huy@example.org | lương 14,000,000 | Python (1 năm)
NV05 | Đỗ Thảo My | Tester | my@example.org | lương 12,000,000 | Kiểm thử API thủ công
Tổng: 5 nhân viên

[02] QUẢN LÝ DỰ ÁN: DANH SÁCH
DA01 | Cổng thông tin nội bộ | Đang thực hiện | 2026-10-01 - 2026-12-31 | 3 thành viên | 3 task
DA02 | Quản lý kho thiết bị | Dự kiến | 2026-11-01 - 2027-01-15 | 3 thành viên | 2 task
Tổng: 2 dự án
```

## 4.2 Giao diện Console – Công việc và Báo cáo

```
[03] CÔNG VIỆC CỦA DỰ ÁN DA01
CV01 | Phân tích yêu cầu | Hoàn thành | Ưu tiên: Cao | 16h | NV03
CV02 | Phát triển API nhân sự | Đang làm | Ưu tiên: Cao | 40h | NV01
CV03 | Kiểm thử API nhân sự | Cần làm | Ưu tiên: Trung bình | 24h | NV02
Tổng: 3 công việc
```

```
[04] THỐNG KÊ HỆ THỐNG
• Thống kê nhân sự: Lập trình viên: 2 | Kiểm thử viên: 2 | Quản lý: 1
• Tiến độ dự án DA01: Task hoàn thành: 1/3 (33.3%)
• Chi phí minh họa (MonthlyTeamCostStrategy): Ngân sách 1 tháng: 56,110,000 VND
  (Chi tiết: Developer NV01 = 16.96M, Tester NV02 = 13.65M, Manager NV03 = 25.50M)
```

## 4.3 Kiểm thử ứng dụng và tính ổn định

| Nhóm tình huống | Cách kiểm tra | Kỳ vọng |
|---|---|---|
| Validation | Email sai, lương âm, ngày ngược, giờ <= 0 | Từ chối nhập, không làm hỏng đối tượng. |
| Dữ liệu trùng | Trùng ID (không phân biệt hoa/thường), trùng email | Ném BusinessRuleException. |
| Phân công | Giao task cho người chưa thuộc dự án | Hệ thống từ chối giao task. |
| Trạng thái | Task DONE chưa có assignee; Project hoàn thành quá sớm | Chặn chuyển trạng thái. |
| Composition | Xóa dự án đang chứa task và thành viên | Task/lượt tham gia bị xóa, Employee giữ lại. |
| Persistence | Lưu CSV rồi khởi động lại hệ thống và load | Số lượng và quan hệ được phục hồi hoàn toàn. |
| CSV hỏng | Ghi sai header của employees.csv | Ném StorageException, không âm thầm bỏ lỗi. |
| Chiến lược | Chạy MonthlyTeamCostStrategy / TaskEffortCostStrategy | Hai công thức cho ra kết quả khác nhau có ý nghĩa. |

# CHƯƠNG 5. ĐÁNH GIÁ & HƯỚNG PHÁT TRIỂN

## 5.1 Ưu điểm, hạn chế và cải tiến

**Ưu điểm:**

• Các entity, service, menu, storage và exception tách thành các package riêng biệt, hỗ trợ bảo trì và giải thích mã nguồn khi vấn đáp.

• Kế thừa và đa hình thể hiện trực tiếp trong thực thi; hai design pattern (Factory, Strategy) được áp dụng đúng ngữ cảnh nghiệp vụ.

• Đủ các nhóm nghiệp vụ tối thiểu, tích hợp các báo cáo thống kê, validation và lưu/đọc dữ liệu CSV bền vững.

**Hạn chế hiện tại:**

• CSV không hỗ trợ đa người dùng cập nhật đồng thời và chưa có cơ chế giao dịch nguyên tử

(atomic transaction) xuyên 4 file.
• UI Console chỉ phù hợp demo/đào tạo; chưa hỗ trợ phân quyền hay tìm kiếm phân trang.

**Một điểm thiết kế chưa tốt và giải pháp:**
• *Vấn đề:* FileDataStore.write() lưu 4 file CSV lần lượt. Nếu bị gián đoạn giữa chừng, dữ liệu giữa các file có thể mất đồng bộ.
• *Giải pháp:* Chuyển sang lưu toàn bộ trạng thái vào một snapshot JSON duy nhất theo cơ chế ghi file tạm → thay thế file (atomic replacement), có checksum hoặc nâng cấp lên database.

## 5.2 Hướng phát triển

• Tăng độ bao phủ JUnit 5, tự động hóa kiểm thử luồng người dùng và bổ sung báo cáo nhân sự theo từng dự án cụ thể.
• Bổ sung cơ chế snapshot file JSON và chức năng sao lưu/khôi phục dữ liệu.

# PHỤ LỤC

## A. Hồ sơ bàn giao, mốc nộp và ghi chú sử dụng

| Hạng mục | Vị trí trong ZIP |
|---|---|
| Mã nguồn Java | src/main/java/vn/pxu/oop/pm/ |
| Kiểm thử | src/test/java/; src/verification/java/ |
| Sơ đồ UML bắt buộc | uml/class-diagram.png; .dot; .puml |
| Dữ liệu mẫu | data/employees.csv; projects.csv; members.csv; tasks.csv |
| Báo cáo | BAO-CAO.pdf / BAO-CAO.docx |
| Bảng phân công | PHAN-CONG.xlsx (phải điền thông tin nhóm thật) |
| Hướng dẫn | README.md, PLAN.md, TASKS.md, docs/DEMO_SCRIPT.md |

**Các mốc tiến độ từ slide:**
• M0: Cuối buổi 1 – Tên nhóm và đề tài dự kiến.
• M1: Trước buổi 3 – Mô tả 1 trang.
• M2: Trước buổi 4 – UML v1.
• M3: Trước buổi 5 – UML v2 và mã khung.
• M4: Một tuần sau buổi 5 – ZIP + GitHub.
• M5: Hai tuần sau buổi 5 – Bảo vệ 20 phút/nhóm (5' trình bày, 5' demo, 8' vấn đáp, 2' nhận xét).